

Assignment: Sub-queries & CTEs

Note: Work inside `college_db.academics` using the tables built in before. Write and test every statement in pgAdmin's Query Tool. For scenario questions, run the steps and observe real results before answering.

Setup Reference

Confirm these tables exist with realistic data (adjust if needed):

Table	Key Columns
academics.departments	dept_id (PK), dept_name
academics.students	student_id (PK), full_name, email, dept_id, cgpa, age
academics.courses	course_id (PK), course_name, dept_id, credits
academics.enrollments	student_id + course_id (PK), enroll_date, grade
academics.employees	emp_id (PK), emp_name, manager_id (self-FK) — from Day 3

Ensure you have: 4+ departments, 8+ students with varied CGPAs, 6+ courses, 12+ enrollments, and the employees hierarchy.

Q1. Scalar Sub-query — Above Average CGPA

Write a query that lists `full_name` and `cgpa` of students whose CGPA is greater than the overall average CGPA of all students. Use a scalar sub-query in the WHERE clause.

-- Write your SELECT statement here:

```
SELECT full_name, cgpa
FROM academics.students
WHERE cgpa > (
    SELECT AVG(cgpa)
    FROM academics.students
);
```

Q2. IN Sub-query — Students with A+ Grade

List the `full_name` of all students who have received at least one 'A+' grade in any course. Use a sub-query with IN.

-- Write your SELECT statement here:

```
SELECT full_name
FROM academics.students
WHERE student_id IN (
    SELECT student_id
    FROM academics.enrollments
    WHERE grade = 'A+'
);
```

Q3. NOT EXISTS — Students with No Enrollments

Using NOT EXISTS (not LEFT JOIN), find all students who have never enrolled in any course. Return student_id and full_name.

-- Write your SELECT statement here:

```
SELECT student_id, full_name
FROM academics.students s
WHERE NOT EXISTS (
    SELECT 1
    FROM academics.enrollments e
    WHERE e.student_id = s.student_id
);
```

Q4. Correlated Sub-query — Topper per Department

For each department, find the student(s) with the highest CGPA in that department. Return full_name, dept_name, and cgpa. Use a correlated sub-query.

-- Write your SELECT statement here:

```
SELECT s.full_name,
       d.dept_name,
       s.cgpa
FROM academics.students s
JOIN academics.departments d
ON s.dept_id = d.dept_id
WHERE s.cgpa = (
    SELECT MAX(s2.cgpa)
    FROM academics.students s2
    WHERE s2.dept_id = s.dept_id
);
```

Q5. Sub-query in FROM (Derived Table)

Write a query that first calculates the average CGPA per department (as a derived table), then returns only those departments whose average CGPA is above 7.0. Show dept_name and avg_cgpa.

-- Write your SELECT statement here:

```
SELECT d.dept_name,
       dt.avg_cgpa
FROM (
    SELECT dept_id,
           AVG(cgpa) AS avg_cgpa
    FROM academics.students
    GROUP BY dept_id
) dt
JOIN academics.departments d
ON dt.dept_id = d.dept_id
WHERE dt.avg_cgpa > 7.0;
```

Q6. EXISTS — Courses Taken by Specific Department

List all courses that have at least one enrollment from a student belonging to the 'Computer Science' department (or whichever dept_name exists in your data). Use EXISTS.

-- Write your SELECT statement here:

```
SELECT c.course_name
FROM academics.courses c
WHERE EXISTS (
    SELECT 1
    FROM academics.enrollments e
    JOIN academics.students s
      ON e.student_id = s.student_id
    JOIN academics.departments d
      ON s.dept_id = d.dept_id
    WHERE e.course_id = c.course_id
    AND d.dept_name = 'Computer Science'
);
```

Q7. Basic CTE — High Achievers Report

Using a CTE named high_achievers (students with cgpa >= 8.0), list their full_name, the courses they enrolled in, and the grade. Order by full_name, course_name.

-- Write your WITH ... SELECT statement here:

```
WITH high_achievers AS (
    SELECT student_id,
           full_name
    FROM academics.students
    WHERE cgpa >= 8.0
)

SELECT h.full_name,
       c.course_name,
       e.grade
FROM high_achievers h
JOIN academics.enrollments e
  ON h.student_id = e.student_id
JOIN academics.courses c
  ON e.course_id = c.course_id
ORDER BY h.full_name, c.course_name;
```

Q8. Multiple CTEs — Department Comparison

Create two CTEs:

- overall_avg → overall average CGPA of all students
- dept_avg → average CGPA per department

Then select departments whose average is higher than the overall average. Show dept_name, dept_avg, and overall_avg.

-- Write your WITH ... SELECT statement here:

```
WITH overall_avg AS (
    SELECT AVG(cgpa) AS overall_avg
    FROM academics.students
),
dept_avg AS (
    SELECT dept_id,
        AVG(cgpa) AS dept_avg
    FROM academics.students
    GROUP BY dept_id
)
```

```
SELECT d.dept_name,
    da.dept_avg,
    oa.overall_avg
FROM dept_avg da
JOIN academics.departments d
ON da.dept_id = d.dept_id
CROSS JOIN overall_avg oa
WHERE da.dept_avg > oa.overall_avg;
```

Q9. CTE + Aggregation — Popular Courses

Using a CTE, find courses that have more than 2 enrollments. Return course_name and enrollment_count, ordered by enrollment_count descending.

-- Write your WITH ... SELECT statement here:

```
WITH popular_courses AS (
    SELECT course_id,
        COUNT(*) AS enrollment_count
    FROM academics.enrollments
    GROUP BY course_id
)
```

```
SELECT c.course_name,
    p.enrollment_count
FROM popular_courses p
JOIN academics.courses c
ON p.course_id = c.course_id
WHERE p.enrollment_count > 2
ORDER BY p.enrollment_count DESC;
```

Q10. Correlated vs JOIN Rewrite

(a) Write a correlated sub-query that finds students whose CGPA is above their own department's average.

(b)

(b) Rewrite the same logic using JOINS (no correlated sub-query).

(c) In 1-2 sentences, which version do you prefer and why?

-- (a), (b), and (c) here:

A)

```
SELECT full_name,
       cgpa
FROM academics.students s
WHERE cgpa >
(
  SELECT AVG(cgpa)
  FROM academics.students s2
  WHERE s2.dept_id = s.dept_id
);
```

B) SELECT s.full_name,

s.cgpa

FROM academics.students s

JOIN (

SELECT dept_id,

AVG(cgpa) AS avg_cgpa

FROM academics.students

GROUP BY dept_id

) d

ON s.dept_id = d.dept_id

WHERE s.cgpa > d.avg_cgpa;

C)

The JOIN version is usually easier to read and often performs better because the department averages are calculated once instead of for every row.

Q11. Recursive CTE — Employee Hierarchy Levels

Using the employees table, write a recursive CTE that shows every employee, their manager's name (or NULL), and their hierarchy level (1 for top-level, 2 for direct reports, etc.). Order by level, emp_name.

-- Write your WITH RECURSIVE ... SELECT here:

WITH RECURSIVE emp_hierarchy AS (

SELECT emp_id,

emp_name,

manager_id,

1 AS level

FROM academics.employees

WHERE manager_id IS NULL

UNION ALL

```
SELECT e.emp_id,  
       e.emp_name,  
       e.manager_id,  
       h.level + 1  
FROM academics.employees e  
JOIN emp_hierarchy h  
  ON e.manager_id = h.emp_id  
)
```

```
SELECT h.emp_name,  
       m.emp_name AS manager_name,  
       h.level  
FROM emp_hierarchy h  
LEFT JOIN academics.employees m  
  ON h.manager_id = m.emp_id  
ORDER BY h.level, h.emp_name;
```

Q12. Complex Multi-step Report

Build a report that answers: "Which students scored an A or A+ in a course that belongs to a different department than their own?" Use CTEs or sub-queries as needed. Return: student_name, student_dept, course_name, course_dept, grade.

-- Write your full query here:

Note: Keep data consistent. Next (Temp Tables & Views) will give you additional tools for organizing intermediate results that you can reuse across sessions.